KSP Assistant

Product Architecture & AI System Report

Project	KSP Assistant (Key Selling Point)
Version	0.1.0
Tech Stack	Next.js 16 + TypeScript + Tailwind v4 + Prisma + PostgreSQL
AI SDK	Anthropic Claude SDK (@anthropic-ai/sdk)
Date	2026-04-03

Document Purpose
This document describes the full architecture of KSP Assistant: user workflow, AI SDK integration, prompt engineering strategy, knowledge base design, and the three-tier content architecture (System Prompt > Series Style Presets > Knowledge Base).

Table of Contents

1. User Workflow Overview

KSP Assistant helps product marketing teams transform raw product specs into structured, tiered selling points with professional packaging copy. The workflow follows a 4-step pipeline:

Step	Tab	Action	Output
1	Parameter Compare (Competitive Spec Compare)	Input own product specs + crawl/paste competitor specs. Compare side-by-side in equal-width columns.	Structured parameter table (own vs N competitors)
2	KSP Tiering (KSP Grading)	AI analyzes parameter advantages/disadvantages. Assign T1 (core), T2 (important), T3 (basic).	Tiered KSP cards with feature names + param values
3	Selling Point Packaging (Copy Packaging)	AI generates 3-layer packaging: L1 Feature Name, L2 Slogan, L3 Sub-points.	Full packaging per KSP item with alternative slogans
4	Refinement (Iteration)	User reviews each card, provides refinement prompts. AI generates new version. Side-by-side compare.	Finalized packaging copy with version history

Key UX Principles:

â¢ Human-in-the-loop at every step. AI generates, human decides.

â¢ Each card is clickable to expand into full detail view for review + refinement.

â¢ Version history preserved. User can compare and revert any version.

â¢ Quick suggestion chips (preset refinement templates) + free-form input.

2. System Architecture

Layer	Technology	Purpose
Frontend	React 19 + Next.js 16 App Router	SSR/CSR hybrid, page routing, API routes
UI	Tailwind CSS v4 + shadcn/ui + lucide-react	Component library, consistent design system
Drag & Drop	@dnd-kit/core + @dnd-kit/sortable	KSP tier card drag-and-drop reordering
State	React useState + Zustand (store)	Client state, i18n, AI settings
Database	PostgreSQL (Neon) + Prisma ORM	12+ models: User, Project, Product, Analysis, KspResult...
Auth	NextAuth v5 + bcryptjs	Email/password + OAuth, session management
AI	Anthropic SDK + OpenAI-compatible adapter	Multi-provider: Claude, GPT-4o, Gemini, MiniMax, Zhipu
Export	pptxgenjs + html2canvas + xlsx	PPT, image, Excel export

3. AI SDK Integration

Multi-Provider Architecture

The system supports 5 AI providers through a unified adapter pattern. Claude uses the native Anthropic SDK; all others use the OpenAI-compatible protocol.

Provider	Default Model	Protocol	Use Case
Claude (Anthropic)	claude-sonnet-4-20250514	Native Anthropic Messages API	Primary: analysis, packaging, agents
OpenAI	gpt-4o	OpenAI Chat Completions	Alternative provider
Google Gemini	gemini-2.5-flash	OpenAI-compatible	Alternative provider
MiniMax	MiniMax-Text-01	OpenAI-compatible	Chinese market optimization
Zhipu AI	glm-4-flash	OpenAI-compatible	Chinese market optimization

Provider Abstraction (src/lib/ai/provider.ts)

Unified interface: chat(messages) and stream(messages) methods. ClaudeAdapter wraps @anthropic-ai/sdk directly. OpenAICompatibleAdapter handles all others via fetch to their OpenAI-compatible endpoints. API keys encrypted in DB (UserAIKey) or provided per-request.

Agent Loop (src/lib/ai/agent-runner.ts)

Generic agent orchestration engine using Anthropic Messages tool_use protocol. Reusable across all 3 agent types. Key interfaces:

Interface	Purpose
AgentToolDef	Tool definition (name, description, input_schema) + async handler function
AgentContext	Shared state: userId, projectId, locale, provider, apiKey, model, onProgress callback, mutable data store
AgentRunnerConfig	systemPrompt, tools[], maxIterations (default 10)
AgentResult	summary text + structured data from context.data

4. Prompt Engineering Architecture

Core Principle: Prompt engineering = fixed few-shot rules hardcoded in system prompts. This is the foundation layer that defines HOW the AI generates content. It does not change per user or per project.

Current Prompt Structure (Packaging)

Layer	Location	Content	Editable?
1. Role Definition	System Prompt (hardcoded)	"Senior product marketing expert specializing in consumer electronics"	No (code change)
2. L1 Rules	System Prompt	High-awareness: param + marketing name. Low-awareness: user-friendly naming.	No (code change)
3. L2 Slogan Rules	System Prompt (slogan-rules.ts)	4-step process: 1. Type selection (emotional/factual/functional) 2. Packaging decision (related features) 3. Extreme word rules 4. Output requirements	No (code change)
4. L3 Techniques	System Prompt (slogan-rules.ts)	3 techniques: Concrete (scenario), Equivalent (comparison), Extreme (edge case)	No (code change)
5. Brand Rules	System Prompt (from DB)	Brand naming conventions, e.g. "battery = Titan Battery". Stored in KnowledgeEntry(entryType="rule").	Yes (DB/UI)
6. Output Format	System Prompt	JSON schema: packagingResults[{l1Name, l2Slogan, l2SloganType, l2Alternatives, l3Details}]	No (code change)
7. Product Context	User Prompt	Product name, segment, competitor advantages	Auto-generated
8. KSP Data	User Prompt	Tier-grouped items: T1/T2/T3 with feature + param value	Auto-generated
9. Knowledge Examples	User Prompt (appended)	Top-3 matching historical examples from Knowledge table. Few-shot reference.	Yes (DB/UI)
10. Refinement	User Prompt (appended)	Current packaging JSON + user adjustment instruction	Yes (per request)

Slogan Generation Rules Detail (slogan-rules.ts)

Step	Rule	Example
1. Type Selection	Has brand story/IP > Emotional Best-in-segment param > Factual Needs translation to benefit > Functional	Emotional: "Dare to Leap" Factual: "Segment's largest 7000mAh" Functional: "2 days, 1 charge"
2. Can Package?	Related features + both T1 > bundle Unrelated > separate	Bundle: battery + charging Separate: chipset + thermal
3. Extreme Words	First in segment > "The first" Best/Largest > "Segment's best" Only one > "Segment's only"	"The first 7000mAh in sub-$200 segment"
4. Output	Short, memorable, one sentence. User remembers at first glance.	"2-Day Titan Battery"

L3 Packaging Techniques

Technique	Description	Example
Concrete (Scenario-based)	Translate abstract params to user-perceivable scenarios	"5000mAh = 12 hours of YouTube streaming"
Equivalent (Comparison)	Compare to familiar objects users know	"1 phone battery = 2 iPhone batteries"
Extreme (Edge case)	Show performance under extreme conditions	"1% battery can still make a 30-min call"

5. Three-Tier Content Architecture

The system uses a three-tier architecture to balance consistency, flexibility, and personalization:

Tier	Name	Scope	Changes How Often	Who Edits
Tier 1 (Foundation)	Prompt Engineering (System Prompt)	Global. Same for all users, all products. Defines the "rules of the game".	Rarely. Only when methodology evolves.	Developer (code change)
Tier 2 (Brand/Series)	Series Style Presets	Per product line. Brand tone, IP persona, user persona, style preferences.	Per product launch cycle. Set once, reuse across SKUs.	Product Manager (UI form)
Tier 3 (Instance)	Knowledge Base (Dynamic Retrieval)	Per feature. Historical examples, competitor references, past successes.	Continuously. Grows with each project.	Anyone (save/upload)

How they compose at generation time:

System Prompt = Tier 1 (fixed rules) + Tier 2 (series preset: tone, IP, constraints)
User Prompt = Product context + KSP data + Tier 3 (matched knowledge examples)
Refinement = User Prompt + current packaging + user instruction

6. Knowledge Base Design

Current Implementation

Category	DB Model	Entry Type	Content	How It Enters
Packaging Examples	Knowledge	packaging	Historical L1/L2/L3 results. Structured JSON with param/selling point/copy.	Click "Save to KB" on Print packaging result
Brand Naming Rules	KnowledgeEntry	rule	Constraints like "battery must be called Titan Battery" or "no superlatives".	Manual input in Knowledge management page
Competitor References	KnowledgeEntry	competitor	Competitor packaging examples. Screenshot + structured analysis.	Manual upload (planned: image parsing)

Retrieval Mechanism

Current: Keyword matching on featureName (e.g., input "battery" matches entries containing "battery"). Top 3 relevant examples are injected as few-shot references in User Prompt. Planned: Vector embedding search for semantic matching (e.g., "battery life" matches "charging speed" examples).

Knowledge Base Roadmap

Phase	Feature	Status
Phase 1	Manual save packaging results to KB + Brand naming rules	Done
Phase 2	Screenshot/image upload for competitor ad references	Planned
Phase 3	User research report PDF upload + parsing	Planned
Phase 4	Vector embedding search (semantic retrieval)	Planned
Phase 5	Auto-archive: auto-save finalized packaging to KB	Planned

7. Series Style Presets (Tier 2)

Series Style Presets sit between the fixed prompt rules (Tier 1) and the dynamic knowledge base (Tier 3). They capture the brand identity and product line personality that should be consistent across all SKUs in a series.

Field	Description	Example
Product Line Name	The series this preset applies to	P Series / GT Series / C Series
Brand Tone of Voice	Adjective keywords defining the brand personality	Young, Bold, Tech-for-all (or: Premium, Sophisticated, Innovative)
Target User Persona	Who buys this product line	18-25 college students / young professionals (or: 30-45 business users)
Price Segment Range	Market positioning	$100-$200 / $200-$400 / $400+
IP Feature Mapping	Parameter to brand IP name mapping	Battery -> Titan Battery Display -> CrystalRes Display Chipset -> Hyper Engine
Copy Style Preference	Preferred packaging techniques, ranked	Scenario-based > Equivalence > Extreme (or: Data-driven > Extreme > Scenario)
Forbidden Expressions	Words/phrases that must NOT appear	"Unmatched", "Kills the competition" (or: market-specific legal constraints)
Reference Tone Examples	Example slogans that match the desired style	"2 Days, 1 Charge" "See Every Detail, Day or Night"

Integration Point: When generating packaging, the selected Series Preset is injected into the System Prompt as additional constraints between the fixed rules (Tier 1) and the output format. This ensures all generated copy aligns with the product line identity.

8. Packaging Refinement & Version Compare

Refinement Workflow

Step	UI Component	What Happens
1. Review	PackagingDetailDialog (card zoom view)	User clicks card to expand. Sees full L1 + L2 + L3 + alternatives. Decides if refinement is needed.
2. Instruct	Refinement Input Area (textarea + chips)	User selects preset chips ("Use scenario-based", "Remove superlatives") or types free-form instruction about their understanding of the feature.
3. Generate	API: /api/ai/packaging (with refinementPrompt)	Current packaging + user instruction sent to AI. AI generates new version that differs meaningfully.
4. Compare	VersionCompareDialog (side-by-side)	Old version (left) vs new version (right). User clicks "Use this version" on preferred side.
5. Iterate	Version History (V1/V2/V3...)	All versions preserved. User can browse history, restore any previous version, or continue refining.

Preset Refinement Templates

Template (ZH)	Template (EN)	What It Does
Switch to scenario-based	Use scenario-based expression	Change L2/L3 to user scenario framing
Remove superlatives	Remove superlatives	Strip extreme/absolute claims
More emotional	More emotional tone	Shift L2 type toward emotional
More data-driven	More data-driven	Shift toward factual with numbers
Add competitor comparison	Add competitor comparison	Reference competitor weaknesses
Make it shorter	Make it shorter & punchier	Compress copy for impact
Use equivalence	Use equivalence comparison	Apply equivalent technique to L3
Emphasize user scenarios	Emphasize user scenarios	Focus L3 on daily use cases

9. Agent System (Claude SDK tool_use)

Three specialized agents handle tasks that benefit from multi-step, autonomous decision-making with tool use:

Agent	Purpose	Tools	Max Iter
Competitive Discovery	Help users find competitor products in a given market/price segment. Scrapes GSMArena, 91mobiles...	1. search_products(query, market) 2. scrape_specs(deviceName, market) 3. recommend_competitors(category, price, market)	15
Review Mining	Analyze product reviews, identify themes, suggest KSP adjustments.	1. analyze_reviews(reviews, productName) 2. deep_dive_theme(theme, reviews) 3. cross_reference_specs(theme, insight) 4. suggest_ksp_adjustments(insights, items)	15
Creative Exploration	Generate and evaluate multiple slogan variants with quality scoring.	1. generate_variants(feature, param, tier) 2. evaluate_variants(variants, brandRules) 3. search_knowledge_base(feature) 4. check_competitor_messaging(variants)	8

Agent Communication: SSE Streaming

All agents stream progress via Server-Sent Events through /api/ai/agent-stream. The useAgentStream React hook consumes these events, providing real-time step updates, progress bars, and abort capability to the UI.

10. Data Model (Key Entities)

Model	Key Fields	Purpose
Project	name, market, segment, launchStatus	Workspace for one product comparison
Product	name, isOwnProduct, params (JSON)	Own product + competitors within a project
Analysis	advantages, disadvantages, neutral (JSON)	Competitive analysis results
KspResult	featureName, tier, l1Name, l2Slogan, l2SloganType, l2Alternatives, l3Details	KSP tier assignment + packaging data
Knowledge	category, brand, content, structured	Legacy KB: packaging examples (few-shot)
KnowledgeEntry	feature, parentFeature, entryType, title, content, structured	Structured KB: packaging / competitor / rule
UserAIKey	provider, encryptedKey	Per-provider encrypted API key storage
UsageLog	action, provider, model, inputTokens, outputTokens, creditsUsed	AI usage tracking + billing
ReviewBatch	totalCount, status, summary	Review analysis batch metadata
ReviewItem	text, sentiment, score, dimensions, highlights	Individual review analysis results

New Type: PackagingVersion (client-side)

Field	Type	Description
version	number	Auto-incrementing version number
l1Name	string	L1 feature name at this version
l2Slogan	string	L2 slogan at this version
l2SloganType	SloganType	factual / functional / emotional
l2Alternatives	SloganAlternative[]?	Alternative slogans
l3Details	L3SubPoint[]?	Sub-point details
refinementPrompt	string?	User instruction that produced this version
createdAt	string	ISO timestamp

11. API Routes

Route	Method	Purpose
/api/ai/agent	POST	Full KSP pipeline (spec fetch > analysis > tiering > packaging)
/api/ai/agent-stream	POST	SSE streaming for Discovery / Review Mining / Creative agents
/api/ai/packaging	POST	Standalone packaging generation (supports refinementPrompt)
/api/ai/analyze	POST	Competitive analysis only
/api/ai/analyze-ksp-tier	POST	Analysis + KSP tiering pipeline
/api/ai/ksp-tier	POST	Standalone KSP tiering
/api/ai/reviews	POST	Review batch analysis
/api/ai/parse-params	POST	Parameter extraction from images/text
/api/projects/[id]	GET/PUT	Project CRUD
/api/projects/[id]/products	POST/PUT/DELETE	Product management within project
/api/projects/[id]/ksp-results	GET/POST	KSP result persistence
/api/knowledge	GET/POST	Knowledge base CRUD
/api/user/ai-keys	GET/POST	AI provider key management
/api/competitor-specs	POST	Competitor spec scraping (GSMArena, 91mobiles)

12. Next Steps & Roadmap

Priority	Feature	Description	Status
P0	Packaging Refinement + Version Comparison	Prior refinement input, preset templates, side-by-side compare, version history	Done
P0	Prompt Architecture Preview	Developer panel showing actual prompt structure sent to AI	Done
P0	Card Detail Dialog	Click-to-zoom card view with full L1/L2/L3 + inline refinement	Done
P1	Series Style Presets	Brand tone, IP mapping, user persona, style preferences per product line	Designed, not built
P1	Knowledge Base: Image Upload	Screenshot competitor ads, parse into structured examples	Planned
P1	Knowledge Base: Brand Guideline PDF	Upload brand guideline PDF, extract rules + examples	Planned
P2	Vector Embedding Search	Semantic KB retrieval (replace keyword matching)	Planned
P2	User Research Import	Upload user research reports, extract insights for KSP refinement	Planned
P2	Auto-Archive	Auto-save finalized packaging to KB as future few-shot examples	Planned
P3	Multi-language Copy	Generate packaging in multiple languages from single source	Planned